南開大學

信息隐藏技术课程实验报告

实验二：语音信号的常用处理方法

学　　院　密码与网络空间安全学院

专　　业　信息安全、法学双学位班

学　　号　2313815

姓　　名　段俊宇

班　　级　信息安全、法学双学位班

一、实验要求

1、 学习智慧课堂：语音信号处理基础。

2、 FFT。

3、 DWT。

4、 DCT。

忽略上述三种变换中的高频成分直至低频成分，分别去除系数的 15%、30%、50%、80%，并做逆变换，然后用 SNR 进行客观评价，观察他们的影响。

二、实验原理

快速傅里叶变换（FFT）：该算法是离散傅里叶变换（DFT）的一种高效算法实现，它能将信号或序列从时域（或空间域）转换到频域，从而展示出信号中包含的不同频率成分及其强度。涉及的公式如下所示：

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N}$$

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{j2\pi kn/N}$$

离散傅里叶变换有如下特点：

- 把时域波形分解成许多不同频率的正弦波的叠加和。
- 傅立叶变换反映的是整个时间段中信号的频谱特性，信号任何时刻的微小变化都会影响到整个频谱。
- 任何有限频段上的信息都无法反映时域的某一个时间段的信号。
- 不适合时变的、非平稳信号。

短时傅里叶变换（SFFT）：对于时变的、非平稳信号,用窗口取得某一时间段的信号，再分析其频谱。公式如下：

$$SFFT_x(t,f) = \int [x(t')g^*(t'-t)] e^{-j2\pi f t'} dt'$$

其中 g*(t)为窗函数。

短时傅里叶变换有如下特点：

- 取较窄的时间窗，可以得到较好的时间分辨率，但是频率分辨率降低。
- 取较宽的时间窗，可以得到较高的频率分辨率，但是时域信号的细节

特性就无法分辨。

- 短时傅立叶变换无法同时满足时间分辨率和频率分辨率的要求。

离散小波变换（DWT）：该算法用于信号处理中的时频分析，它通过对基本小波函数进行离散化的伸缩和平移，将信号分解成一系列不同尺度和位置的小波系数。小波变换在低频部分具有较高的频率分辨率，较低的时间分辨率；在高频部分具有较高的时间分辨率，较低的频率分辨率，符合低频信号变化缓慢而高频信号变化迅速的特点。其主要公式为：

$$W(a,b)=\frac{1}{\sqrt{a}}\int x(t)\psi^{*}\left(\frac{t-b}{a}\right)dt$$

其中，$\psi(t)$为母小波，a 为尺度因子，b 为平移因子。通过离散化 a 和 b，可实现对信号的高效分解与重构。

离散余弦变换（DCT）：该算法是一种将信号从空间域或时域转换到频域的变换方法，其核心思想是使用不同频率的余弦函数作为基函数。正变换公式为：

$$T(u)=\sum_{x=0}^{N-1}f(x)g(x,u)\quad u=0,1,2,\dots,N-1$$

其中正变换核有下面两种：

$$g(x,0)=\frac{1}{\sqrt{N}}\quad g(x,u)=\sqrt{\frac{2}{N}}cos\frac{(2x+1)u\pi}{2N}$$

DCT 变换的系数有下面两种：

$$C(0)=\frac{1}{\sqrt{N}}\sum_{x=0}^{N-1}f(x)$$

$$C(u)\sqrt{\frac{2}{N}}\sum_{x=0}^{N-1}f(x)\,cos\frac{(2x+1)u\pi}{2N}\quad u=1,2,\dots N-1$$

反变换公式为：

$$f(x)=\frac{1}{\sqrt{N}}C(0)+\sqrt{\frac{2}{N}}\sum_{\mathrm{u}=1}^{N-1}C(u)\,cos\frac{(2x+1)u\pi}{2N}\quad x=0,1,\dots N-1$$

三、实验环境

Matlab_R2025b，Signal Processing Toolbox，Wavelet Toolbox

四、实验过程

1. FFT 变换处理函数

如下所示，我将使用注释的方式对代码进行解释。

```
% FFT 变换处理函数
function [x_reconstructed, snr_val] = fft_analysis(x, ratio)
    X = fft(x);
    N = length(X);
    keep_count = floor(N * (1 - ratio));
    half_keep = floor( keep_count / 2);
    % 保留低频，去除高频
    X_filtered = X;
    X_filtered(half_keep+1:N-half_keep) = 0;
    % 逆变换
    x_reconstructed = real(ifft(X_filtered));
    % 计算 SNR
    snr_val = snr(x, x - x_reconstructed);
end
```

这部分代码保留了 1-ratio 的系数，也就是低频部分，之后将中间的高频部分置为 0，这样就达到了去除相应比例高频成分的效果。

2. DWT 变换处理函数

如下所示，我将使用注释的方式对代码进行解释。

```
% DWT 变换处理函数
function [x_reconstructed, snr_val] = dwt_analysis(x, ratio)
    [c, l] = wavedec(x, 3, 'db4');
    % 按系数大小排序，去除小系数
    sorted_c = sort(abs(c), 'descend');
    threshold_idx = floor(length(c) * (1 - ratio));
    threshold = sorted_c(threshold_idx);
    % 阈值处理
    c_filtered = c;
    c_filtered(abs(c) < threshold) = 0;
    % 逆变换
    x_reconstructed = waverec(c_filtered, l, 'db4');
    % 计算 SNR
    snr_val = snr(x, x - x_reconstructed);
end
```

这部分代码先对原始音频进行三级小波变换，然后按照系数大小排序并去除小系数，将 threshold_idx 编号对应的系数作为阈值，然后将绝对值小

于阈值的系数置为 0，实现对高频的去除。

3. DCT 变换处理函数

如下所示，我将使用注释的方式对代码进行解释。

```
% DCT 变换处理函数
function [x_reconstructed, snr_val] = dct_analysis(x, ratio)
    Xdct = dct(x);
    N = length(Xdct);
    keep_count = floor(N * (1 - ratio));
    % 保留低频，去除高频
    Xdct_filtered = Xdct;
    Xdct_filtered(keep_count+1:end) = 0;

    x_reconstructed = idct(Xdct_filtered);
    snr_val = snr(x, x - x_reconstructed);
end
```

这部分代码将 DCT 系数从低频到高频排列，保留了 1-ratio 的系数部分，其余部分为高频，将其置为 0，实现对高频的去除。

4. 主函数

```
filename = 'test.mp3';
if exist(filename, 'file')
    [x, fs] = audioread(filename);
    x = mean(x, 2);  % 转单声道
    % 使用音频的前 10 秒
    target_samples = 10 * fs;
    if length(x) > target_samples
        start_idx = floor((length(x) - target_samples) / 2);
        x = x(start_idx+1:start_idx+target_samples);
    else
        x = [x; zeros(target_samples-length(x), 1)];
    end
    t = (0:length(x)-1)/fs;
end

% 定义去除比例
removal_ratios = [0.15, 0.30, 0.50, 0.80];
ratio_names = {'15%', '30%', '50%', '80%'};
ratio_percent = removal_ratios * 100;

% 初始化存储
snr_fft = zeros(size(removal_ratios));
```

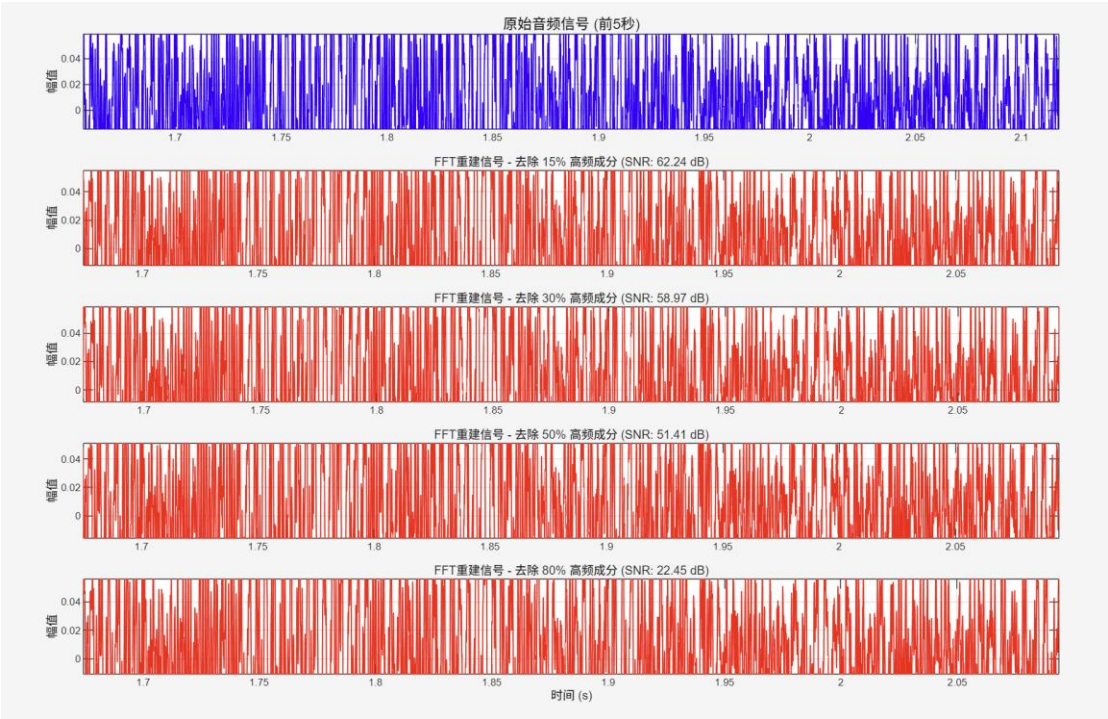
```
snr_dwt = zeros(size(removal_ratios));
snr_dct = zeros(size(removal_ratios));

x_fft_all = cell(length(removal_ratios), 1);
x_dwt_all = cell(length(removal_ratios), 1);
x_dct_all = cell(length(removal_ratios), 1);

% 获取三种变换后的数据
for i = 1:length(removal_ratios)
    ratio = removal_ratios(i);
    [x_fft_all{i}, snr_fft(i)] = fft_analysis(x, ratio);
    [x_dwt_all{i}, snr_dwt(i)] = dwt_analysis(x, ratio);
    [x_dct_all{i}, snr_dct(i)] = dct_analysis(x, ratio);
end

% 波形图展示 5 秒的内容
display_duration = min(5, length(x)/fs);
display_samples = round(display_duration * fs);

% 图表展示
fig = figure('Name', '信号变换分析', 'Position', [100, 100, 1400, 900]);
tabgroup = uitabgroup('Parent', fig);

tab1 = uitab('Parent', tabgroup, 'Title', 'SNR 对比');
ax1 = axes('Parent', tab1, 'Position', [0.1, 0.1, 0.8, 0.8]);

plot(ax1, ratio_percent, snr_fft, 'b-o', 'LineWidth', 2, 'MarkerSize', 8);
hold(ax1, 'on');
plot(ax1, ratio_percent, snr_dwt, 'g-s', 'LineWidth', 2, 'MarkerSize', 8);
plot(ax1, ratio_percent, snr_dct, 'r-^', 'LineWidth', 2, 'MarkerSize', 8);
hold(ax1, 'off');

grid(ax1, 'on');
xlabel(ax1, '去除比例 (%)', 'FontSize', 12);
ylabel(ax1, 'SNR (dB)', 'FontSize', 12);
title(ax1, 'FFT、DWT、DCT 变换去除高频成分后的 SNR 对比', 'FontSize', 14);
legend(ax1, 'FFT', 'DWT', 'DCT', 'Location', 'best');
set(ax1, 'FontSize', 11);

for i = 1:length(ratio_percent)
    text(ax1, ratio_percent(i), snr_fft(i)+0.5, sprintf('%.1f', snr_fft(i)), ...
        'HorizontalAlignment', 'center', 'FontSize', 9);
    text(ax1, ratio_percent(i), snr_dwt(i)+0.5, sprintf('%.1f', snr_dwt(i)), ...
        'HorizontalAlignment', 'center', 'FontSize', 9);
```

```
    text(ax1, ratio_percent(i), snr_dct(i)+0.5, sprintf('%.1f', snr_dct(i)), ...
        'HorizontalAlignment', 'center', 'FontSize', 9);
end

% 三种变换后重建信号的波形图与原始图像对比
tab2 = uitab('Parent', tabgroup, 'Title', 'FFT 信号对比');

subplot(5, 1, 1, 'Parent', tab2);
plot(t(1:display_samples), x(1:display_samples), 'b-', 'LineWidth', 1);
title('原始音频信号 (前 5 秒)', 'FontSize', 12);
ylabel('幅值', 'FontSize', 10);
grid on;
xlim([0, display_duration]);

% 显示不同去除比例的重建信号
for i = 1:length(removal_ratios)
    subplot(5, 1, i+1, 'Parent', tab2);
    plot(t(1:display_samples), x_fft_all{i}(1:display_samples), 'r-', 'LineWidth', 1);
    title(sprintf('FFT 重建信号 - 去除 %s 高频成分 (SNR: %.2f dB)', ratio_names{i}, snr_fft(i)), 'FontSize', 10);
    ylabel('幅值', 'FontSize', 10);
    grid on;
    xlim([0, display_duration]);
    if i == length(removal_ratios)
        xlabel('时间 (s)', 'FontSize', 10);
    end
end

tab3 = uitab('Parent', tabgroup, 'Title', 'DWT 信号对比');

subplot(5, 1, 1, 'Parent', tab3);
plot(t(1:display_samples), x(1:display_samples), 'b-', 'LineWidth', 1);
title('原始音频信号 (前 5 秒)', 'FontSize', 12);
ylabel('幅值', 'FontSize', 10);
grid on;
xlim([0, display_duration]);

for i = 1:length(removal_ratios)
    subplot(5, 1, i+1, 'Parent', tab3);
    plot(t(1:display_samples), x_dwt_all{i}(1:display_samples), 'r-', 'LineWidth', 1);
    title(sprintf(['DWT 重建信号 - 去除 %s 高频成分' ...
        ' (SNR: %.2f dB)'], ratio_names{i}, snr_dwt(i)), 'FontSize', 10);
```

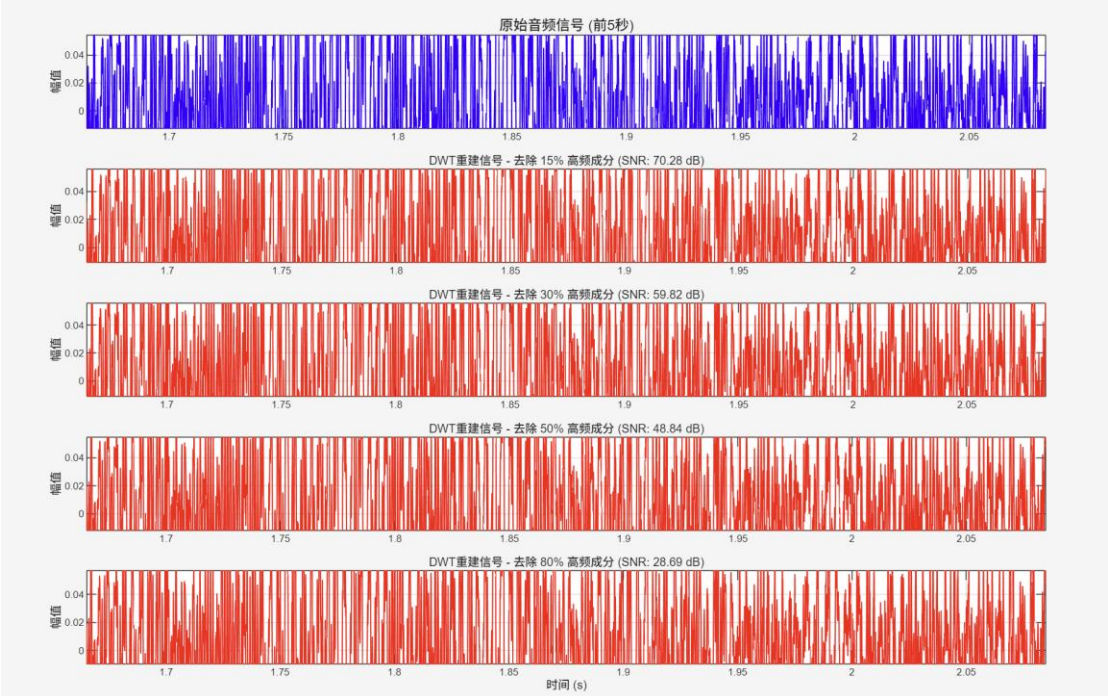
```
    ylabel('幅值', 'FontSize', 10);
    grid on;
    xlim([0, display_duration]);
    if i == length(removal_ratios)
        xlabel('时间 (s)', 'FontSize', 10);
    end
end

tab4 = uitab('Parent', tabgroup, 'Title', 'DCT 信号对比');

subplot(5, 1, 1, 'Parent', tab4);
plot(t(1:display_samples), x(1:display_samples), 'b-', 'LineWidth', 1);
title('原始音频信号 (前 5 秒)', 'FontSize', 12);
ylabel('幅值', 'FontSize', 10);
grid on;
xlim([0, display_duration]);

for i = 1:length(removal_ratios)
    subplot(5, 1, i+1, 'Parent', tab4);
    plot(t(1:display_samples), x_dct_all{i}(1:display_samples), 'r-', 'LineWidth', 1);
    title(sprintf('DCT 重建信号 - 去除 %s 高频成分 (SNR: %.2f dB)', ratio_names{i}, snr_dct(i)), 'FontSize', 10);
    ylabel('幅值', 'FontSize', 10);
    grid on;
    xlim([0, display_duration]);
    if i == length(removal_ratios)
        xlabel('时间 (s)', 'FontSize', 10);
    end
end
```

在本次实验中我选择了一段赛车引擎和漂移的声音作为样本，将其命名为 test.mp3。由于这部分音频的声音主要集中在前 10 秒，因此主函数中选取音频文件的前 10 秒作为测试样本。接下来调用前面的三种处理函数对音频分别进行处理，最后使用折线图和波形图进行结果对比。

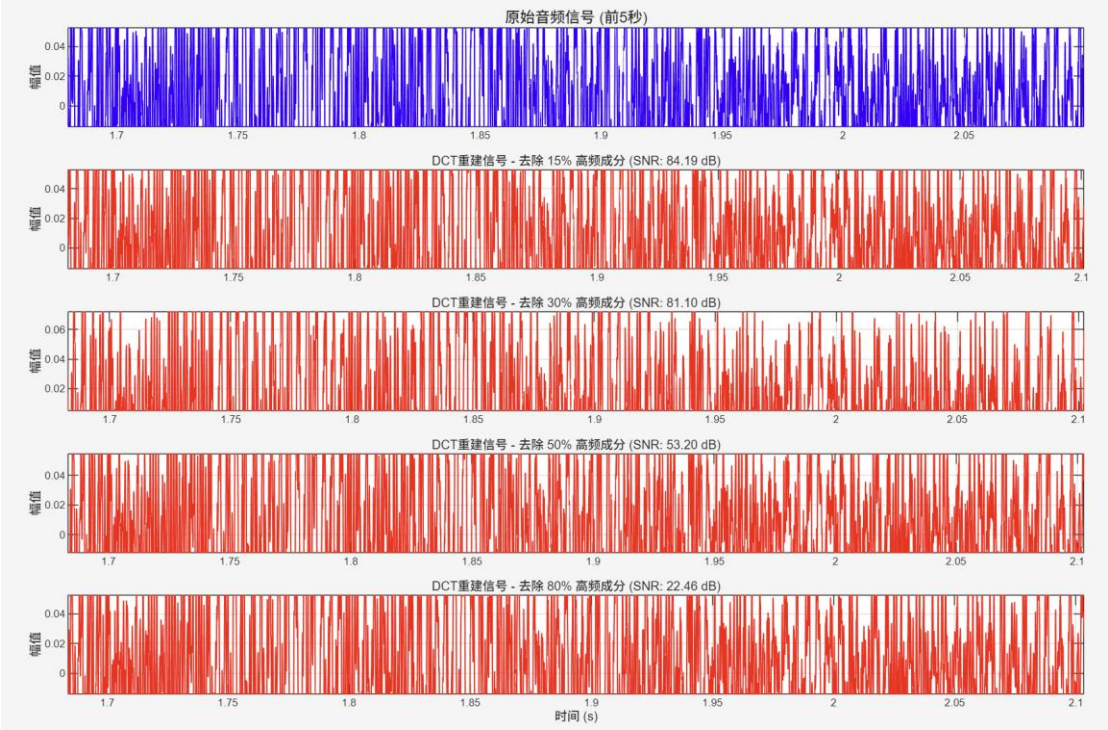
如下所示是使用 FFT 对高频部分按不同比例处理后的 SNR 以及原始信号对比图像：



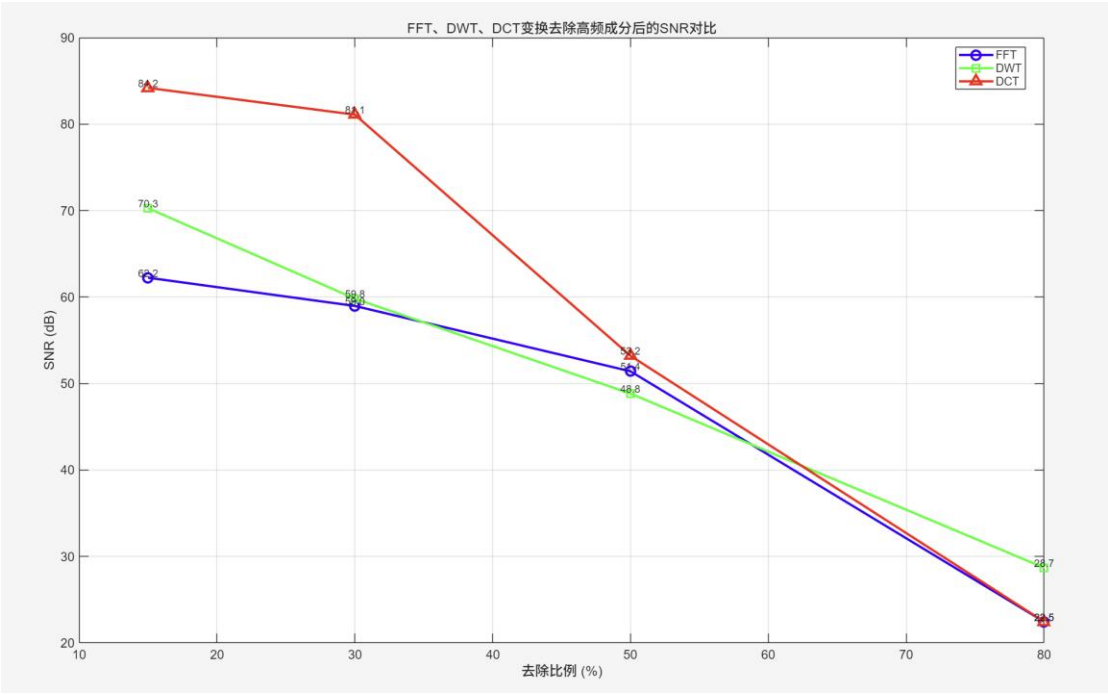
如下所示是使用 DWT 对高频部分按不同比例处理后的 SNR 以及原始信号对比图像：



如下所示是使用 DCT 对高频部分按不同比例处理后的 SNR 以及原始信号对比图像：



下面是三种变换去除不同比例高频信号后的 SNR 对比图：



从上面的实验结果可以看出，波形图大体来看基本相同，只有放大后有细微的差别，这说明在保证可接受质量的前提下，通过去除高频信号实现音频压缩是可行的。三种变换去除不同比例高频信号的 SNR 对比图较为直观，从中能够看出随着高频信号去除比例的增加，SNR 数值会显著下降，这样的音频相较原始音频会显得沉闷，体现了高频信号对音频清晰度和细节的贡献。

五、实验结论及心得体会

本次实验中我巩固了课上学习的语音信号的常用处理方法，在实验过程中使用 Matlab 调用工具库，实现了三种变换对原始音频的处理，并按不同比例去除高频信号。通过对重建信号的 SNR 进行对比，验证了音频压缩的可行性和高频信号对信号保真度的贡献，巩固了课上学习的知识。